

DATA PREPROCESSING

```
[63]: import pandas as pd
import seaborn as sns

#Load data set
df=sns.load_dataset("titanic")

print(df.head())
print(df.shape) #rows and columns
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	\
0	0	3	male	22.0	1	0	7.2500	S	Third	
1	1	1	female	38.0	1	0	71.2833	C	First	
2	1	3	female	26.0	0	0	7.9250	S	Third	
3	1	1	female	35.0	1	0	53.1000	S	First	
4	0	3	male	35.0	0	0	8.0500	S	Third	
	who	adult_male	deck	embark_town	alive	alone				
0	man	True	NaN	Southampton	no	False				
1	woman	False	C	Cherbourg	yes	False				
2	woman	False	NaN	Southampton	yes	True				
3	woman	False	C	Southampton	yes	False				
4	man	True	NaN	Southampton	no	True				

(891, 15)

```
[64]: df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
#   Column      Non-Null Count  Dtype
---  -
0   survived    891 non-null    int64
1   pclass      891 non-null    int64
```

2	sex	891	non-null	object
3	age	714	non-null	float64
4	sibsp	891	non-null	int64
5	parch	891	non-null	int64
6	fare	891	non-null	float64
7	embarked	889	non-null	object
8	class	891	non-null	category
9	who	891	non-null	object
10	adult_male	891	non-null	bool
11	deck	203	non-null	category
12	embark_town	889	non-null	object
13	alive	891	non-null	object
14	alone	891	non-null	bool

dtypes: bool(2), category(2), float64(2), int64(4), object(5)
memory usage: 80.7+ KB

```
[65]: #1. Inconsistent formats      #same data but different format and identifies them.
      #Concept :
      #The same value is represented in different formats.
      #Create Problem
```

```
[66]: df.loc[0, 'embark_town']='southampton'
      df.loc[1, 'embark_town']='SOUTHAMPTON'
      df.loc[2, 'embark_town']='Southampton'
      print(df['embark_town'].unique())
```

```
['southampton' 'SOUTHAMPTON' 'Southampton' 'Queenstown' 'Cherbourg' nan]
```

```
[67]: df['embark_town']=df['embark_town'].str.title()      #change the first letter to captial
```

```
[68]: print(df['embark_town'].unique())      #it shows only the unique values
```

```
['Southampton' 'Queenstown' 'Cherbourg' nan]
```

```
[69]: print(df['embark_town'].isnull().sum())
```

```
2
```

```
[70]: df['embark_town']=df['embark_town'].fillna('Kingstown') #it replace the nan with Kingstone
```

```
[71]: print(df['embark_town'].unique())

['Southampton' 'Queenstown' 'Cherbourg' 'Kingstown']
```

```
[72]: #2. Duplicate Rows
#A Duplicate row is an exact copy of another row.
#Create Duplicate Records
```

```
[73]: duplicate_rows=df.iloc[:5]
df_dup=pd.concat([df, duplicate_rows])
print(df_dup.shape)

(896, 15)
```

```
[74]: #Find Duplicate
print(df_dup.duplicated().sum())

112
```

```
[75]: #View Duplicate Rows
df_dup[df_dup.duplicated()]
print(df.head())
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	\
0	0	3	male	22.0	1	0	7.2500	S	Third	
1	1	1	female	38.0	1	0	71.2833	C	First	
2	1	3	female	26.0	0	0	7.9250	S	Third	
3	1	1	female	35.0	1	0	53.1000	S	First	
4	0	3	male	35.0	0	0	8.0500	S	Third	

	who	adult_male	deck	embark_town	alive	alone
0	man	True	NaN	Southampton	no	False
1	woman	False	C	Southampton	yes	False
2	woman	False	NaN	Southampton	yes	True
3	woman	False	C	Southampton	yes	False

```
[76]: df_dup=df_dup.drop_duplicates()
      print(df_dup.duplicated().sum())
```

0

```
[77]: #3. Typos
      #Spelling mistakes create unnecessary categories
      #Create Type
```

```
[78]: df.loc[0, 'embark_town']='Southamton'    #spelling mistake count
      df.loc[1, 'embark_town']='Southhampton'
```

```
[79]: print(df['embark_town'].value_counts())
```

```
embark_town
Southampton    643
Cherbourg       167
Queenstown      77
Kingstown        2
Southamton       1
Southhampton     1
Name: count, dtype: int64
```

```
[80]: #Fix Typos
      df['embark_town']=df['embark_town'].replace({
          'Southamton':'Southampton',
          'Southhampton':'Southampton'
      })
```

```
#Verify
```

```
print(df['embark_town'].value_counts())
```

```
embark_town
Southampton    645
Cherbourg       167
```

```
Cherbourg      167
Queenstown     77
Kingstown       2
Name: count, dtype: int64
```

```
[81]: #Detect possible Typos
print(sorted(df['embark_town'].dropna().unique()))

['Cherbourg', 'Kingstown', 'Queenstown', 'Southampton']
```

#4. Unit Mismatches #Values representing the same measurement are sorted in different units

Create Problem #Suppose some ages are accidently entered in months intead of years

```
[83]: df.loc[0, 'age'] = 240
df.loc[1, 'age'] = 300
```

```
[84]: #Detect Unusual Values
df['age'].describe()
```

```
[84]: count      714.000000
mean         30.371387
std          19.362012
min           0.420000
25%          20.125000
50%          28.000000
75%          38.750000
max          300.000000
Name: age, dtype: float64
```

```
[85]: #Fix Unit Mismatch
#Convert month to years
df.loc[df['age'] > 120, 'age'] = (
    df.loc[df['age'] > 120, 'age'] / 12 #chaning from moth to years
)
```



```
)

#Verify
df['age'].describe()
```

```
[85]: count    714.000000
      mean     29.678109
      std     14.525902
      min       0.420000
      25%     20.000000
      50%     28.000000
      75%     38.000000
      max     80.000000
      Name: age, dtype: float64
```

```
[86]: #5. Wrong Category Labels
      #df['survived']
      #Contains:
      #0
      #1
      #Not every readable.
```

```
[87]: df['survived_label']=df['survived'].map({
      0:'No',
      1:'Yes'
      })
      print(df.head())
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	\
0	0	3	male	20.0	1	0	7.2500	S	Third	
1	1	1	female	25.0	1	0	71.2833	C	First	
2	1	3	female	26.0	0	0	7.9250	S	Third	
3	1	1	female	35.0	1	0	53.1000	S	First	
4	0	3	male	35.0	0	0	8.0500	S	Third	

	who	adult_male	deck	embark_town	alive	alone	survived_label
0	man	True	NaN	Southampton	no	False	No
1	woman	False	C	Southampton	yes	False	Yes
2	woman	False	NaN	Southampton	yes	True	Yes
3	woman	False	C	Southampton	yes	False	Yes
4	man	True	NaN	Southampton	no	True	No

```
[88]: #Redundant Columns
      #Columns with little or no useful information

      #Example :
      #df['deck']
      #Contains mostly missing values.
```

```
[89]: #check:
      df.isnull().mean()
```

```
[89]: survived          0.000000
      pclass           0.000000
      sex              0.000000
      age              0.198653
      sibsp            0.000000
      parch            0.000000
      fare             0.000000
      embarked         0.002245
      class            0.000000
      who              0.000000
      adult_male        0.000000
      deck              0.772166
      embark_town       0.000000
      alive             0.000000
      alone             0.000000
      survived_label     0.000000
      dtype: float64
```

```
[90]: df.columns
```

```
[90]: Index(['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch', 'fare',
          'embarked', 'class', 'who', 'adult_male', 'deck', 'embark_town',
          'alive', 'alone', 'survived_label'],
          dtype='object')
```

```
[92]: df.drop(columns=['deck'], inplace=True)
df.head()
```

```
[92]:
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	embark_town	alive	alone	survived_label
0	0	3	male	20.0	1	0	7.2500	S	Third	man	True	Southampton	no	False	N
1	1	1	female	25.0	1	0	71.2833	C	First	woman	False	Southampton	yes	False	Y
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	Southampton	yes	True	Y
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	Southampton	yes	False	Y
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	Southampton	no	True	N

```
[ ]: #Handling Null Values in Titanic dataset
#What are Null Values?
#Null Values(missing values)
```

```
[93]: df.isnull().sum()
```

```
[93]: survived      0
pclass            0
sex               0
age              177
sibsp            0
parch            0
fare             0
```


File Edit View Insert Cell Help Python [conda env:base] * Anaconda Toolbox

```
embarked    2
class       0
who         0
adult_male  0
embark_town 0
alive       0
alone       0
survived_label
dtype: int64
```

```
[54]: #Method 1 :Mean Imputation
```

```
#Replace missing values with the average value.
df['age']=df['age'].fillna(df['age'].mean())
df.head(20)
```

```
[54]: survived  pclass    sex      age  sibsp  parch    fare  embarked  class  who  adult  male  embark town  alive  alone  sur
```

0	0	3	male	20.000000	1	0	7.2500	S	Third	man	True	Southampton	no	False
1	1	1	female	25.000000	1	0	71.2833	C	First	woman	False	Southampton	yes	False
2	1	3	female	26.000000	0	0	7.9250	S	Third	woman	False	Southampton	yes	True
3	1	1	female	35.000000	1	0	53.1000	S	First	woman	False	Southampton	yes	False
4	0	3	male	35.000000	0	0	8.0500	S	Third	man	True	Southampton	no	True
5	0	3	male	29.678109	0	0	8.4583	Q	Third	man	True	Queenstown	no	True
6	0	1	male	54.000000	0	0	51.8625	S	First	man	True	Southampton	no	True
7	0	3	male	2.000000	3	1	21.0750	S	Third	child	False	Southampton	no	False
8	1	3	female	27.000000	0	2	11.1333	S	Third	woman	False	Southampton	yes	False
9	1	2	female	14.000000	1	0	30.0708	C	Second	child	False	Cherbourg	yes	False
10	1	3	female	4.000000	1	1	16.7000	S	Third	child	False	Southampton	yes	False

10	1	3	female	4.000000	1	1	16.7000	S	Third	child	False	Southampton	yes	False
11	1	1	female	58.000000	0	0	26.5500	S	First	woman	False	Southampton	yes	True
12	0	3	male	20.000000	0	0	8.0500	S	Third	man	True	Southampton	no	True
13	0	3	male	39.000000	1	5	31.2750	S	Third	man	True	Southampton	no	False
14	0	3	female	14.000000	0	0	7.8542	S	Third	child	False	Southampton	no	True
15	1	2	female	55.000000	0	0	16.0000	S	Second	woman	False	Southampton	yes	True
16	0	3	male	2.000000	4	1	29.1250	Q	Third	child	False	Queenstown	no	False
17	1	2	male	29.678109	0	0	13.0000	S	Second	man	True	Southampton	yes	True
18	0	3	female	31.000000	1	0	18.0000	S	Third	woman	False	Southampton	no	False
19	1	3	female	29.678109	0	0	7.2250	C	Third	woman	False	Cherbourg	yes	True

```
[94]: #Method 2 : Median
#Replace missing values with the middle value
#Example df['age_median']=df['age'].fillna(df['age'].median())

#Replace missing values with the average value.
df['age']=df['age'].fillna(df['age'].median())
df.head(20)
```

	survived	pclass	sex	age	sibsp	parth	fare	embarked	class	who	adult_male	embark_town	alive	alone	survived_
0	0	3	male	20.0	1	0	7.2500	S	Third	man	True	Southampton	no	False	
1	1	1	female	25.0	1	0	71.2833	C	First	woman	False	Southampton	yes	False	
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	Southampton	yes	True	
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	Southampton	yes	False	
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	Southampton	no	True	
5	0	3	male	28.0	0	0	8.4583	Q	Third	man	True	Queenstown	no	True	
6	0	1	male	54.0	0	0	51.8625	S	First	man	True	Southampton	no	True	
7	0	3	male	2.0	3	1	21.0750	S	Third	child	False	Southampton	no	False	
8	1	3	female	27.0	0	2	11.1333	S	Third	woman	False	Southampton	yes	False	
9	1	2	female	14.0	1	0	30.0708	C	Second	child	False	Cherbourg	yes	False	
10	1	3	female	4.0	1	1	16.7000	S	Third	child	False	Southampton	yes	False	
11	1	1	female	58.0	0	0	26.5500	S	First	woman	False	Southampton	yes	True	
12	0	3	male	20.0	0	0	8.0500	S	Third	man	True	Southampton	no	True	
13	0	3	male	39.0	1	5	31.2750	S	Third	man	True	Southampton	no	False	
14	0	3	female	14.0	0	0	7.8542	S	Third	child	False	Southampton	no	True	
15	1	2	female	55.0	0	0	16.0000	S	Second	woman	False	Southampton	yes	True	
16	0	3	male	2.0	4	1	29.1250	Q	Third	child	False	Queenstown	no	False	
17	1	2	male	28.0	0	0	13.0000	S	Second	man	True	Southampton	yes	True	
18	0	3	female	31.0	1	0	18.0000	S	Third	woman	False	Southampton	no	False	
19	1	3	female	28.0	0	0	7.2250	C	Third	woman	False	Cherbourg	yes	True	

```
[ ]: #Let's check Age distribution
#df['age'].skew()
#If skewness high:
#Use median
#If data is nearly normal:
#Use mean
```

```
[95]: #Method 3 : Mode Imputation
#Replace missing values with most frequent value
#df['embarked_mode']=df['embarked'].fillna(df['embarked'].mode()[0])

#Replace missing values with the average value.
df['age']=df['age'].fillna(df['age'].mode())
df.head(20)
```

[95]:	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	embark_town	alive	alone	survived
0	0	3	male	20.0	1	0	7.2500	S	Third	man	True	Southampton	no	False	
1	1	1	female	25.0	1	0	71.2833	C	First	woman	False	Southampton	yes	False	
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	Southampton	yes	True	
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	Southampton	yes	False	
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	Southampton	no	True	
5	0	3	male	28.0	0	0	8.4583	Q	Third	man	True	Queenstown	no	True	
6	0	1	male	54.0	0	0	51.8625	S	First	man	True	Southampton	no	True	
7	0	3	male	2.0	3	1	21.0750	S	Third	child	False	Southampton	no	False	
8	1	3	female	27.0	0	2	11.1333	S	Third	woman	False	Southampton	yes	False	
9	1	2	female	14.0	1	0	30.0708	C	Second	child	False	Cherbourg	yes	False	
10	1	3	female	4.0	1	1	16.7000	S	Third	child	False	Southampton	yes	False	

11	1	1	female	58.0	0	0	26.5500	S	First	woman	False	Southampton	yes	True
12	0	3	male	20.0	0	0	8.0500	S	Third	man	True	Southampton	no	True
13	0	3	male	39.0	1	5	31.2750	S	Third	man	True	Southampton	no	False
14	0	3	female	14.0	0	0	7.8542	S	Third	child	False	Southampton	no	True
15	1	2	female	55.0	0	0	16.0000	S	Second	woman	False	Southampton	yes	True
16	0	3	male	2.0	4	1	29.1250	Q	Third	child	False	Queenstown	no	False
17	1	2	male	28.0	0	0	13.0000	S	Second	man	True	Southampton	yes	True
18	0	3	female	31.0	1	0	18.0000	S	Third	woman	False	Southampton	no	False
19	1	3	female	28.0	0	0	7.2250	C	Third	woman	False	Cherbourg	yes	True

```
[104]: #Method 4 : Forward Fill(FFill)
#Use the previous value to fill missing data
#Before imputataion
```

```
[105]: print(df['embarked'].isnull().sum())

2
```

```
[108]: #Forward fill
df['embarked_ffill']=df['embarked'].ffill()
#Check missing values
print(df['embarked_ffill'].isnull().sum())

0
```

```
[109]: df.columns
```

```
[109]: Index(['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch', 'fare',
```



```
[111]: #Method 5 : KNN Imputation
#K-Nearest Nighbors(KNN) imputation fills missing values using information from similar rows
#Instead of using a single value(mean/median), it finds the K most similar records and uses their values

import pandas as pd
import numpy as np

data={
    'age': [22,25,np.nan,24,23],
    'fare': [7.25,8.05,7.90,8.00,7.50]
}
df=pd.DataFrame(data)
print(df)
```

	age	fare
0	22.0	7.25
1	25.0	8.05
2	NaN	7.90
3	24.0	8.00
4	23.0	7.50

```
[114]: #Apply KNN Imputation
from sklearn.impute import KNNImputer

KNN=KNNImputer(n_neighbors=3)

df_KNN=pd.DataFrame(
    KNN.fit_transform(df),
    columns=df.columns
)
print(df_KNN) #the nan is been replaced by the value for this we use KNN

#sklearn - Scikit-Learn Library for machine Learning and data preprocessing
#impute - Module for handling missing values
#KNNImputer - Class that fills missing values using the K-Nearest Neighbors(KNN)
```

```
#fit_transform(df) - Learns from the data and fills missing values
#convert the result back into a pandas DataFrame.
```

```

    age  fare
0  22.0  7.25
1  25.0  8.05
2  24.0  7.90
3  24.0  8.00
4  23.0  7.50

```

```

[116]: #Check Missing values
import pandas as pd
import seaborn as sns
from sklearn.impute import KNNImputer

#Load dataset
df=sns.load_dataset('titanic')

#forward Fill for embarked
df['embarked_ffill']=df['embarked'].ffill()

#KNN Imputation for embarked
cols=[
    'age',
    'fare',
    'sibsp',
    'parch',
    'pclass'
]
imputer=KNNImputer(n_neighbors=5)

df[cols]=imputer.fit_transform(df[cols])

print(df.isnull().sum())

```

```

survived      0
pclass       0
sex           0
age           0
sibsp        0
parch        0
fare         0
embarked      2
class         0
who           0
adult_male    0
deck         688
embark_town   2
alive         0
alone         0
embarked_ffill 0
dtype: int64

```

```

[ ]: #Backward fill
df.fillna(method='bfill',inplace=True)

or
df.bfill(inplace=True)

```

▼ Encoding Using Titanic Dataset ¶

```

[ ]: #What is Encoding?
#Machine Learning model work with numbers,not text

#Example:
#Sex
#male
#female

```

```
#male
#A modal cannot directly understand "male" and "female".
#We convert the categories into numbers using Encoding.
```

Types of Encoding

Types of Encoding 1.Label Encoding Ordinal data - ordered data , nominal data - not ordered Assign a number to each category

Example: Size Small Medium High becomes size 0 1 2 #the order matters

When to use Label encoding: Ordinal Categories

Example: Small<Medium<large low<medium<high Poor<Average<Good

Nominal Categories: Example: Red,Blue,Green Male , Female India,USA,UK

2. One-Hot Encoding(Nominal Data) Creates a seperate binary column for each category

When to use One-Hot Encoding: Nominal Categories Example: Country,Gender,Color,City,Department

3. Target Encoding Replace category with average value of the target variable

Why use target encoding? Imagine: 5000 Cities One-Hot Encoding would create: 5000 Columns Not practical.

Target Encoding Creates

1. Column

```
[121]: import pandas as pd
import seaborn as sns
from sklearn.preprocessing import LabelEncoder
```

```
#Label Encoding
le=LabelEncoder()

df['sex_encoded']=le.fit_transform( #directly we can able to change to sex
    df['sex']
)

#One-Hot Encoding
df=pd.get_dummies(
    df,
    columns=['embarked'],
)

#Target Encoding
target_map=df.groupby('pclass')['survived'].mean() #using pclass to seperate survived,find avg and fill)['survived'].m

df['class_target']=(df['pclass'].map(target_map))

print(df.head())
```

	survived	pclass	sex	age	sibsp	parch	fare	class	who	\
0	0	3	male	22.0	1	0	7.2500	Third	man	
1	1	1	female	38.0	1	0	71.2833	First	woman	
2	1	3	female	26.0	0	0	7.9250	Third	woman	
3	1	1	female	35.0	1	0	53.1000	First	woman	
4	0	3	male	35.0	0	0	8.0500	Third	man	

	adult_male	deck	embark_town	alive	alone	sex_encoded	embarked_C	\
0	True	NaN	Southampton	no	False	1	False	
1	False	C	Cherbourg	yes	False	0	True	
2	False	NaN	Southampton	yes	True	0	False	
3	False	C	Southampton	yes	False	0	False	
4	True	NaN	Southampton	no	True	1	False	

	embarked_Q	embarked_S	class_target
0	False	True	0.242363
1	False	False	0.629630
2	False	True	0.242363
3	False	True	0.629630
4	False	True	0.242363

Feature Scaling Using Titanic Dataset

```
[ ]: What is Feature Scaling?
      Feature Scaling transforms numerical features to a similar scale

Why it is needed?
  In the titanic Dataset
Ex:
  Feature Range Age 0-80 Fare 0-512

Here, Fare has a much larger range than Age.

Many ML Algorithms calculate distances, so Fare can dominate the model.

1. MinMaxScaler
  Scales values between 0 and 1

Advantages
  Preserves shape of sistribution
  Values always between 0 and 1
  Useful for Neural Network

Disadvantage
  Sensitive to outliers
```


2. StandardScaler

Centres data around mean=0 and standard deviation=1

Advantage

- Works well for normally distributed data
- Commonly used
- Required by many algorithms.

Best for

Logistic Regression, SVM, PCA, KNN, Linear Regression

Disadvantages

- Still affected by extreme outliers.

3. RobustScaler

Uses median and IQR instead of Mean and Standard Deviation

Advantages

- Handles Outliers well
- Better for skewed distributions

Disadvantages

- Values are not limited to 0-1

```
[125]: from sklearn.preprocessing import MinMaxScaler

scaler=MinMaxScaler()

df[['age_minmax','fare_minmax']]=scaler.fit_transform( #two columns are created
    df[['age','fare']] #changing to 0's to 1's
)

df[['age_minmax','fare_minmax']].head()
```

```
)
df[['age_minmax', 'fare_minmax']].head()
```

```
[125]:
```

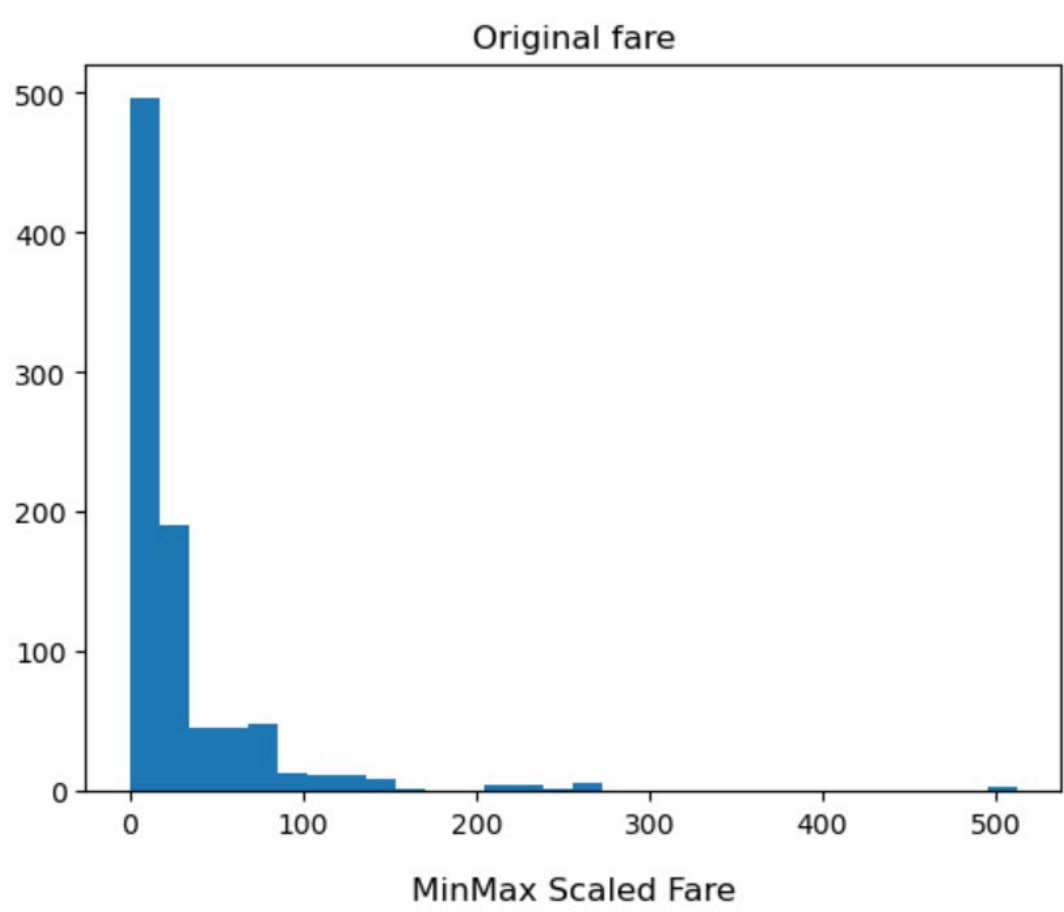
	age_minmax	fare_minmax
0	0.271174	0.014151
1	0.472229	0.139136
2	0.321438	0.015469
3	0.434531	0.103644
4	0.434531	0.015713

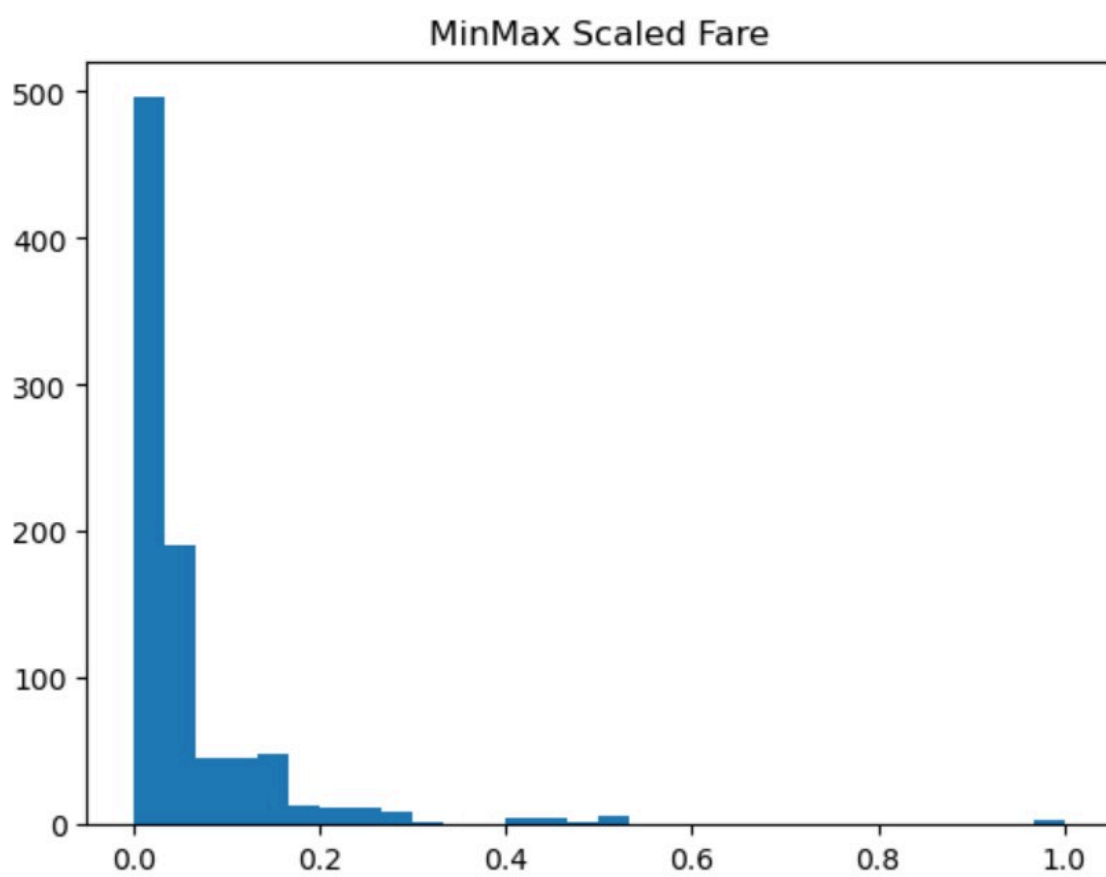
```
[131]: import matplotlib.pyplot as plt

plt.hist(df['fare'].dropna(), bins=30)
plt.title("Original fare")
plt.show()

plt.hist(df['fare_minmax'], bins=30)
plt.title("MinMax Scaled Fare")
plt.show()
```

```
plt.show()
```





```
[133]: from sklearn.preprocessing import StandardScaler

scaler=StandardScaler()

df[['age_std','fare_std']]=scaler.fit_transform( #two columns are created
    df[['age','fare']] #changing to 0's to 1's
)

df[['age_std','fare_std']].head()
```

```
[133]:
```

	age_std	fare_std
0	-0.530377	-0.502445
1	0.571831	0.786845
2	-0.254825	-0.488854
3	0.365167	0.420730
4	0.365167	-0.486337

```
[135]: from sklearn.preprocessing import RobustScaler

scaler=RobustScaler()

df[['age_robust','fare_robust']]=scaler.fit_transform( #two columns are created
    df[['age','fare']] #changing to 0's to 1's
)

df[['age_robust','fare_robust']].head()
```

```
[135]:
```

	age_robust	fare_robust
0	-0.335664	-0.312011

4 0.365167 -0.486337

```
[135]: from sklearn.preprocessing import RobustScaler

scaler=RobustScaler()

df[['age_robust','fare_robust']]=scaler.fit_transform( #two columns are created
    df[['age','fare']] #changing to 0's to 1's
)

df[['age_robust','fare_robust']].head()
```

```
[135]:
```

	age_robust	fare_robust
0	-0.335664	-0.312011
1	0.559441	2.461242
2	-0.111888	-0.282777
3	0.391608	1.673732
4	0.391608	-0.277363